

Instituição: Centro Educacional da Fundação Salvador Arena (CEFSA / FESA)

Curso: Engenharia da Computação

Integrantes do Grupo:

- Eduardo Urbanovicz de Souza – RA: 082230018
- Lucas Daiki Honda Kuniyosi – RA: 082230020
- Ronaldo de Oliveira Santos – RA: 082230031

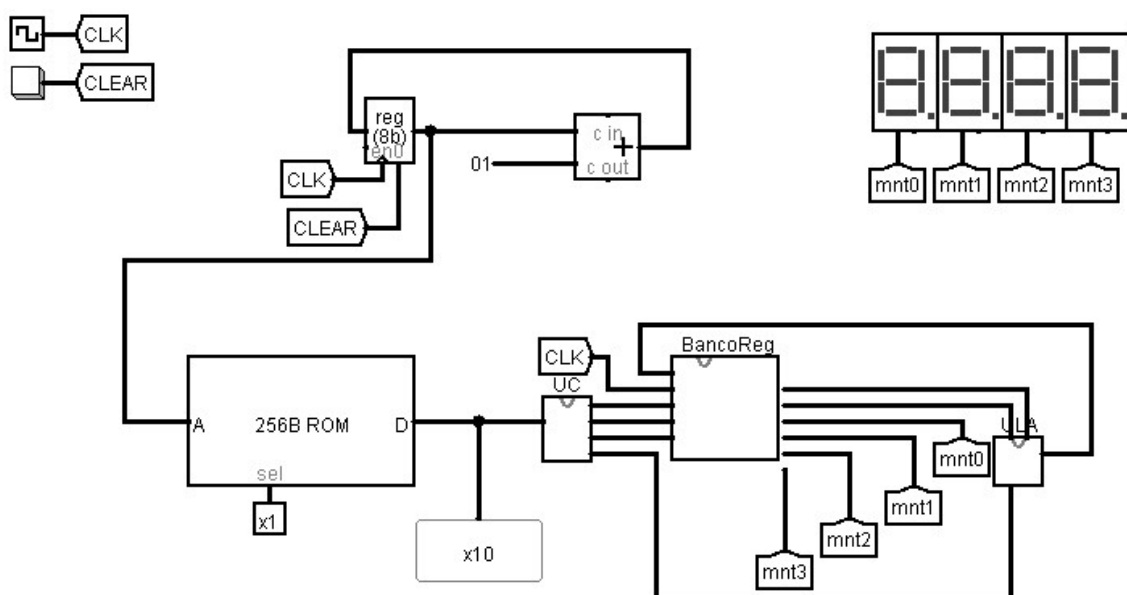
Professor: Vinícius S. Borges

1. Objetivo do Projeto

O objetivo deste projeto é projetar, simular e validar uma Unidade Central de Processamento (CPU) de 4 bits utilizando o software Logisim. O escopo abrange a aplicação prática de conceitos fundamentais de Arquitetura de Computadores, exigindo a criação de um conjunto de instruções (ISA) customizado e a segregação estrita entre o Caminho de Dados (Datapath) e a Unidade de Controle, operando de forma síncrona.

2. Arquitetura da CPU (Diagrama de Blocos)

A CPU possui uma arquitetura focada em processamento sequencial sem desvios, com um Caminho de Dados de 4 bits e um barramento de instrução de 10 bits. O fluxo de dados é cíclico, onde os operandos saem do Banco de Registradores, são processados na ULA e retornam pelo barramento de *Write-back* para o próprio Banco.



3. Descrição dos Módulos

ULA (Unidade Lógica e Aritmética)

A ULA é o núcleo de processamento do circuito, operando estritamente em 4 bits. É estruturada em torno de um multiplexador de 8 canais de entrada. As portas de 0 a 3 são conectadas a blocos somadores e subtratores (operações aritméticas: Soma, Subtração, Incremento, Decremento). As portas de 4 a 7 recebem os resultados de portas lógicas nativas (AND, OR, XOR, NOT). A operação ativa é roteada para a saída através de um sinal seletor de 3 bits.

Banco de Registradores

Responsável pelo armazenamento temporário. Composto por quatro registradores físicos de 4 bits (R0, R1, R2, R3).

- **Via de Escrita:** Protegida por um decodificador (1 para 4) acoplado aos pinos de *Enable* (Habilitação). Isso garante que o dado retornado pela ULA seja gravado exclusivamente no registrador de destino no momento do pulso de *clock*.
- **Via de Leitura:** Utiliza dois multiplexadores independentes, permitindo a leitura assíncrona e simultânea de dois registradores diferentes para alimentar os operandos da ULA.

Unidade de Controle

Circuito puramente combinacional. Recebe a instrução bruta de 10 bits vinda da memória e utiliza a técnica de fatiamento (*splitting*) para distribuir os sinais. Direciona os endereços de registradores para os multiplexadores e decodificador do Banco de Registradores, e executa uma lógica de agrupamento entre o *Opcode* e o campo *Funct* para gerar o código seletor final de 3 bits exigido pela ULA.

Memória

Utiliza-se uma memória ROM para armazenamento fixo do programa de máquina (instruções). Possui barramento de endereço de 8 bits (endereçoado pelo Program Counter - PC, permitindo até 256 linhas de código) e barramento de dados configurado obrigatoriamente para 10 bits, correspondendo ao tamanho exato da instrução do processador.

4. Descrição Detalhada do Conjunto de Instruções

A arquitetura (ISA) desenvolvida possui instruções de tamanho fixo (10 bits) divididas em 5 campos:

- **Opcode [Bits 9-8]:** Categoria da operação (00 = Aritmética, 01 = Lógica).
- **Destino (R1) [Bits 7-6]:** Endereço de gravação do resultado.
- **Operando 1 (R2) [Bits 5-4]:** Primeiro valor lido.

- **Operando 2 (R3) [Bits 3-2]:** Segundo valor lido.
- **Funct [Bits 1-0]:** Operação específica.

Tabela de Operações Implementadas:

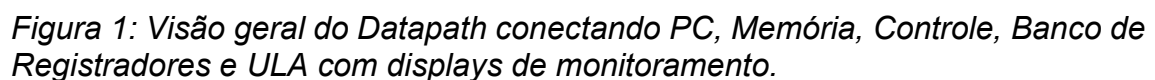
Instrução	Opcode	Funct	Operação Realizada
ADD	00	00	Soma dois registradores
SUB	00	01	Subtrai R3 de R2
INC	00	10	Incrementa R2 em +1
DEC	00	11	Decrementa R2 em -1
AND	01	00	Operação E lógica bit a bit
OR	01	01	Operação OU lógica bit a bit
XOR	01	10	Operação OU Exclusivo bit a bit
NOT	01	11	Inverte todos os bits de R2

5. Funcionamento do Ciclo de Instrução

O hardware processa cada instrução em três etapas simultâneas e encadeadas, validadas pelo pulso de *clock* global:

1. **Fetch (Busca):** O endereço apontado pelo Program Counter (PC) seleciona uma linha na ROM. A instrução de 10 bits é enviada ao barramento principal.

- ## 6. Prints do Circuito e da Simulação



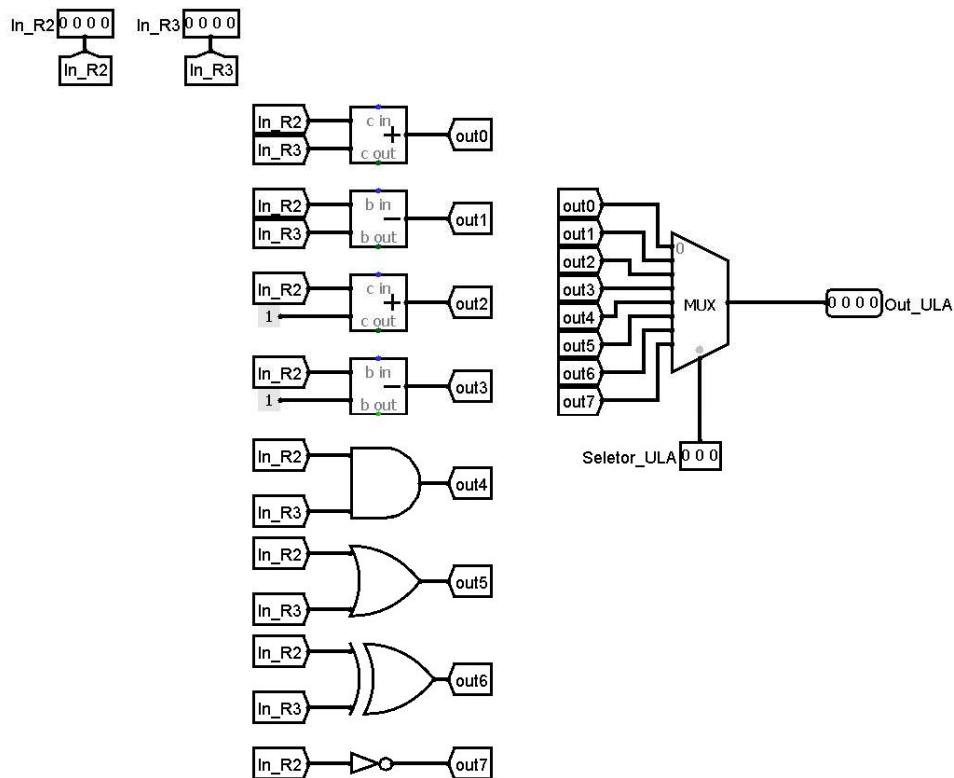


Figura 2: Subcircuito da ULA demonstrando o roteamento lógico e o multiplexador de saída.

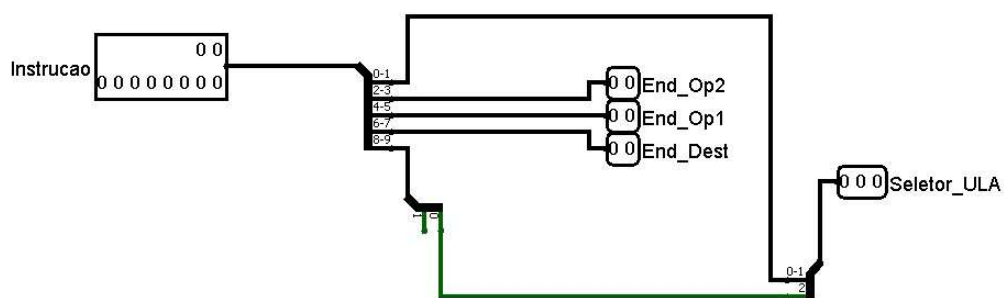


Figura 3: Subcircuito da Unidade de Controle executando o fatiamento da instrução de 10 bits.

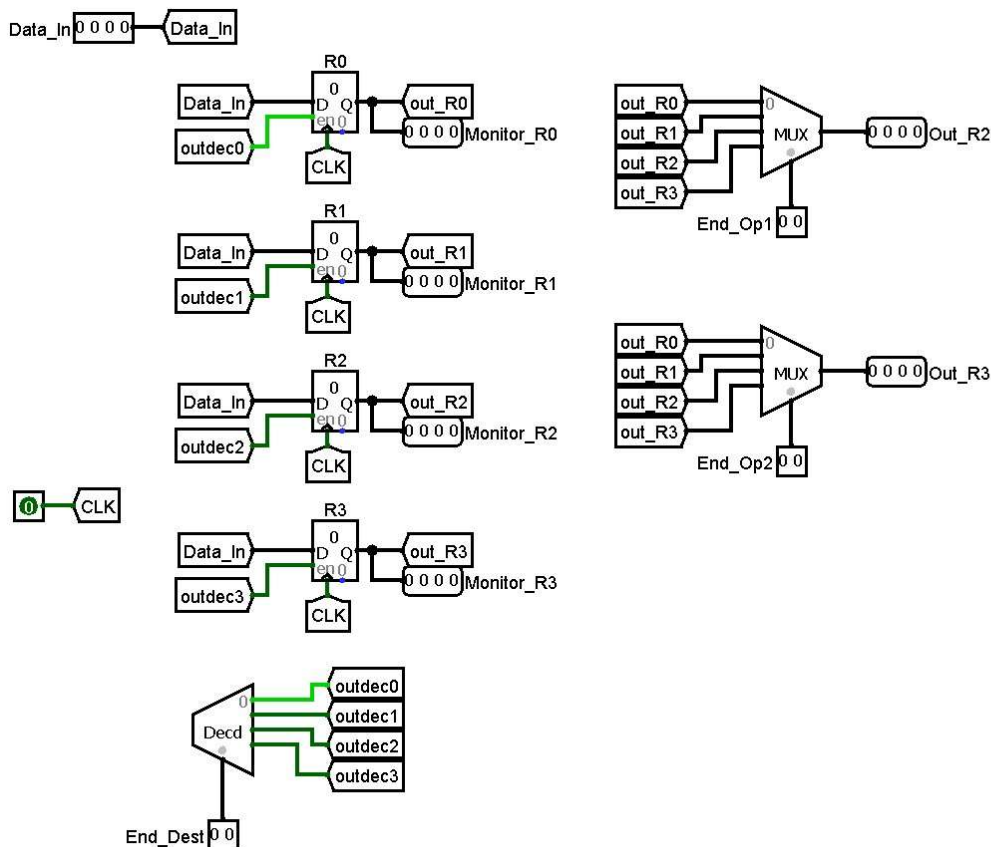


Figura 4: Subcircuito do Banco de Registradores com o isolamento de escrita (decodificador) e vias de leitura simultâneas.

7. Execução de um Programa de Teste

Devido à ausência de instruções de Entrada/Saída ou de *Load* direto da memória, a validação funcional da CPU foi realizada por meio de um algoritmo de geração interna de dados, focado em calcular a **Sequência de Fibonacci**. O programa utiliza a instrução de Incremento para iniciar o dado e, em seguida, opera somas em cascata, demonstrando o funcionamento íntegro das vias de leitura e gravação.

Código de Máquina Carregado na ROM (Hexadecimal):

042 084 0d8 02c 070 084 0d8

Mapeamento da Execução:

- **PC 00 (042):** INC R0 -> R1. Resultado: R1 = 1.
- **PC 01 (084):** ADD R0, R1 -> R2. Resultado: R2 = 1.
- **PC 02 (0D8):** ADD R1, R2 -> R3. Resultado: R3 = 2.
- **PC 03 (02C):** ADD R2, R3 -> R0. Resultado: R0 = 3.
- **PC 04 (070):** ADD R3, R0 -> R1. Resultado: R1 = 5.
- **PC 05 (084):** ADD R0, R1 -> R2. Resultado: R2 = 8.
- **PC 06 (0D8):** ADD R1, R2 -> R3. Resultado: R3 = 13 (Exibido como **D** no display hexadecimal).

8. Conclusão e Possíveis Melhorias

O projeto atingiu todos os objetivos propostos. A arquitetura modular desenvolvida no Logisim funcionou sem falhas de roteamento, conflitos de impedância ou atrasos de propagação incorretos. O processador foi capaz de decodificar e executar operações aritméticas e lógicas com precisão no ciclo de *clock* adequado.

Durante o desenvolvimento e execução do programa de teste, ficou evidente o limite físico imposto pela arquitetura de 4 bits. No ciclo posterior ao PC 06 do programa de Fibonacci, ao tentar somar 8 e 13, o resultado real (21) ultrapassa o limite de armazenamento de 4 bits, gerando um descarte do bit mais significativo (*Carry-out*) e registrando o valor truncado (5). Trata-se de um comportamento físico padronizado e correto para este hardware, conhecido como *overflow*.

Possíveis Melhorias (Trabalhos Futuros):

1. **Implementação de Memória RAM:** Adição de instruções estruturais de *LOAD* e *STORE*, permitindo o acesso a um banco de memória estendido para leitura e gravação de variáveis que superem o limite dos 4 registradores internos.
2. **Registrador de Status (Flags):** Inserção de um registrador para armazenar bits de condição gerados pela ULA (como *Carry*, *Overflow*, *Zero* e *Negative*), garantindo a integridade dos dados e avisando o sistema quando a capacidade da via for extrapolada.
3. **Instruções de Desvio (Branching):** Alteração da unidade de incremento do PC para suportar desvios condicionais e incondicionais (ex: *JUMP*), permitindo a execução de estruturas de controle de repetição (*loops*), o que tornaria o processador Turing-completo.